

The Mathematics of width: 100%

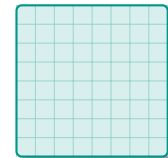
Why scaling low-resolution images forces extreme pixelation

🖼️ Native Image File

Low resolution (e.g., 100×100 px)
Contains exactly 10,000
unique pixels of data.

📱 Web Container

High-density Display (e.g., 400×400 px)
Requires 160,000 pixels to fill the space.



$W_{native} = 100px$

width: 100%;

The Command:
"Stretch my 100px of data
to fill this 400px void."

$W_{container} = 400px$



The Mathematical Deficit

- Scale Factor (1D):** $S = \frac{W_{container}}{W_{native}} = \frac{400px}{100px} = 4$
- Area Multiplier (2D):** Since images are 2D grids, the total pixel requirement scales quadratically:
 $Area = S^2 = 4^2 = 16$
- The Gap:** The browser needs 160,000 pixels, but only has 10,000 real pixels. It must **invent 15 fake pixels** for every 1 real pixel.



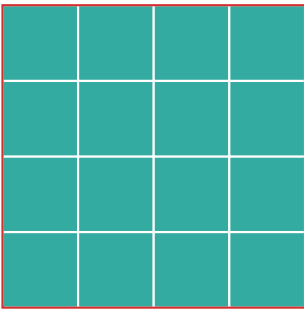
How Browsers "Invent" Pixels (Interpolation Algorithms)

Original Data
(1 Pixel)



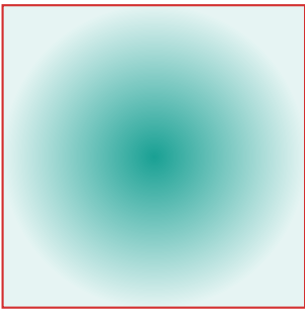
1 real pixel of
source data.

Nearest Neighbor
(CSS: image-
rendering: pixelated)



Duplicates the original pixel color
exactly. Causes "blocky" artifacts.

Bicubic
(Default Browser
Behavior)



Averages surrounding col-
ors. Causes "muddy"
or blurry artifacts.